

Chapter 10

SFT Best Practices and Techniques

Supervised Fine-Tuning (SFT) is the foundation of the RLHF pipeline. The quality of the SFT model determines the ceiling of what RL can achieve: RL can refine and improve a behaviour, but it cannot reliably introduce a behaviour that is entirely absent from the SFT model. This section covers the key techniques for effective SFT.

10.1 Sequence Packing for Efficiency

The Padding Problem

Standard SFT batches pad all sequences to the length of the longest sequence in the batch. For datasets with high length variance (e.g., a mix of short instructions and long documents), this wastes 50–80% of compute on padding tokens. Sequence packing eliminates this waste.

Sequence packing concatenates multiple short examples into a single sequence of length `max_seq_length`, separated by EOS tokens. The attention mask ensures that tokens from different examples do not attend to each other:

1. Sort examples by length (optional, improves packing efficiency).
2. Greedily pack examples into bins of size `max_seq_length`.
3. Use a block-diagonal attention mask to prevent cross-example attention.
4. Compute loss only on non-padding tokens.

Packing Efficiency

- Typical packing efficiency: 85–95% (vs 20–50% with padding).
- Speedup: 2–4× for datasets with high length variance.
- Memory: similar to padding (same total tokens per batch).
- Caveat: requires careful attention masking to avoid cross-contamination.

Sequence Packing in TRL

```
from trl import SFTConfig, SFTTrainer

config = SFTConfig(
    max_seq_length=4096,
    packing=True,           # enable sequence packing
    output_dir="sft_model",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-5,
    num_train_epochs=3,
```

```
)

trainer = SFTTrainer(
    model=model,
    args=config,
    train_dataset=dataset,
    # dataset_text_field="text", # or use formatting_func
)
trainer.train()
```

10.2 Chat Templates and Formatting

Why Chat Templates Matter

Language models are trained on raw text, but instruction-following models need to distinguish between system prompts, user messages, and assistant responses. Chat templates encode this structure into the token sequence. Using the wrong template (or no template) at inference time causes significant performance degradation.

ChatML Format

ChatML is the most widely used chat template:

```
# ChatML format
template = "<|im_start|>system
{system_message}<|im_end|>
<|im_start|>user
{user_message}<|im_end|>
<|im_start|>assistant
{assistant_message}<|im_end|>"""
```

Llama Format

Llama 3 uses a different template with special tokens:

```
# Llama 3 format
template = ""<|begin_of_text|><|start_header_id|>system<|end_header_id|>
{system_message}<|eot_id|><|start_header_id|>user<|end_header_id|>
{user_message}<|eot_id|><|start_header_id|>assistant<|end_header_id|>
{assistant_message}<|eot_id|>"""
```

Applying Chat Templates in TRL

```
from transformers import AutoTokenizer
from trl import SFTConfig, SFTTrainer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

def formatting_func(example):
    """Apply chat template to a dataset example."""
    messages = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": example["instruction"]},
        {"role": "assistant", "content": example["response"]},
    ]
    return tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=False,
    )
```

```

config = SFTConfig(
    max_seq_length=2048,
    output_dir="sft_model",
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    args=config,
    train_dataset=dataset,
    formatting_func=formatting_func,
)

```

10.3 Completion-Only Masking

Why Mask the Prompt?

In instruction fine-tuning, the model should learn to generate the assistant's response, not to predict the user's question or the system prompt. Computing loss on the prompt tokens wastes gradient signal and can cause the model to “memorise” prompts rather than learning to respond to them. Completion-only masking sets the loss to zero for all non-assistant tokens.

Completion-Only Masking in TRL

```

from trl import SFTConfig, SFTTrainer, DataCollatorForCompletionOnlyLM
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

# Define the response template (tokens after which loss is computed)
response_template = "<|start_header_id|>assistant<|end_header_id|>"
collator = DataCollatorForCompletionOnlyLM(
    response_template=response_template,
    tokenizer=tokenizer,
)

config = SFTConfig(
    max_seq_length=2048,
    output_dir="sft_model",
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    args=config,
    train_dataset=dataset,
    data_collator=collator,  # completion-only masking
    formatting_func=formatting_func,
)

```

Completion Masking Pitfalls

- The response template must exactly match the tokenised form. Off-by-one errors in tokenisation can cause the mask to be applied incorrectly.
- For very short responses, masking the prompt may leave too few tokens for meaningful gradient signal. Consider a minimum response length threshold.
- Multi-turn conversations require masking all non-assistant turns, not just the first.

10.4 Data Mixing Strategies for Multi-Task SFT

The Multi-Task Challenge

Training on multiple tasks simultaneously can improve generalisation but also causes *task interference*: gradients from different tasks conflict, degrading performance on individual tasks. Data mixing strategies control the relative contribution of each task to the training signal.

Proportional Mixing

Sample from each dataset proportionally to its size:

$$p_k = \frac{N_k}{\sum_{j=1}^K N_j},$$

where N_k is the number of examples in dataset k . This is the default in most frameworks and works well when datasets are of similar quality.

Temperature Mixing

Apply a temperature T to smooth the proportions:

$$p_k \propto N_k^{1/T}.$$

$T = 1$: proportional mixing. $T \rightarrow \infty$: uniform mixing. $T < 1$: over-samples large datasets. $T > 1$: over-samples small datasets.

Quality-Weighted Mixing

Weight datasets by estimated quality (e.g., perplexity under a reference model, human quality ratings):

$$p_k \propto N_k \cdot q_k,$$

where q_k is the quality score for dataset k .

Data Mixing in TRL

```
from datasets import concatenate_datasets, interleave_datasets

# Proportional mixing (default)
mixed_dataset = concatenate_datasets([
    dataset_math,
    dataset_code,
    dataset_general,
]).shuffle(seed=42)

# Temperature mixing (T=2: over-sample small datasets)
mixed_dataset = interleave_datasets(
    [dataset_math, dataset_code, dataset_general],
    probabilities=[0.4, 0.4, 0.2], # manually set after temperature scaling
    seed=42,
)

config = SFTConfig(output_dir="sft_model")
trainer = SFTTrainer(
    model=model,
    args=config,
    train_dataset=mixed_dataset,
)
```

10.5 When SFT Hurts – Catastrophic Forgetting and Alignment Tax

As LLMs transition through sequential training phases — pre-training → continued pre-training → SFT → RLHF/DPO — performance degradation frequently manifests on standard benchmarks. Two **fundamentally distinct** phenomena drive these regressions, and confusing them leads to wrong mitigation strategies.

10.5.1 Catastrophic Forgetting (Structural Erasure)

Catastrophic Forgetting

Catastrophic forgetting is an **unintentional optimization failure**: when a network optimized on distribution $\mathcal{D}_A$ is subsequently trained on a disjoint distribution $\mathcal{D}_B$, the weight updates required for $\mathcal{D}_B$ *physically overwrite* the parameter structures encoding $\mathcal{D}_A$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_B(\theta_t) \implies \mathcal{L}_A(\theta_{t+1}) \gg \mathcal{L}_A(\theta_t) \quad (10.1)$$

The knowledge is **destroyed** — the weights encoding Task A no longer exist. This is irreversible without retraining.

Symptoms:

- Complete breakdown on tasks not in fine-tuning data (e.g., model forgets how to do math after SFT on chat data)
- Loss of language diversity — model only generates in the narrow style of fine-tuning distribution
- Reduced factual accuracy on knowledge not reinforced during fine-tuning
- Degraded multilingual ability after English-only SFT

Mechanistic cause — Fisher Information perspective: The Fisher Information Matrix F of Task A identifies which parameters are “important” for $\mathcal{D}_A$:

$$F = \mathbb{E}_{x \sim \mathcal{D}_A} \left[\nabla_{\theta} \log \pi_{\theta}(x) \nabla_{\theta} \log \pi_{\theta}(x)^T \right] \quad (10.2)$$

Parameters with high Fisher eigenvalues are critical for Task A. Unconstrained gradient descent on Task B ignores these eigenvalues entirely — $\Delta\theta$ points along $\nabla \mathcal{L}_B$ regardless of whether it destroys high-Fisher directions for $\mathcal{L}_A$.

10.5.2 Alignment Tax (Behavioral Constraint)

The alignment tax is a **deliberate, expected trade-off**: the model’s raw capability (unconstrained generation, maximal reasoning bandwidth) decreases because the policy is constrained to produce safe, well-formatted, preference-aligned outputs.

Mechanism: During DPO/PPO, the policy π_{θ} is penalized for deviating from the reference π_{ref} via KL divergence:

$$r_{\text{implicit}}(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \quad (10.3)$$

This leash constrains the model’s **output distribution** — it cannot explore high-variance reasoning paths that deviate too far from the reference. The knowledge is **not erased**; it’s *suppressed*. The model still “knows” the answer but its distribution is flattened toward safe, generic responses.

Symptoms:

- Over-refusal (“I can’t help with that” for benign queries)
- Stylistic stiffness — hedge words, excessive caveats, verbose safety disclaimers

- Lower scores on raw capability benchmarks (MMLU, HumanEval) while improving on preference benchmarks (MT-Bench, AlpacaEval)
- Reduced ability to produce complex, high-entropy outputs (creative writing, novel algorithms)

10.5.3 Comparative Taxonomy

Table 10.1: Catastrophic Forgetting vs. Alignment Tax — complete comparison.

Dimension	Catastrophic Forgetting	Alignment Tax
Intentionality	Unintentional (optimization artifact)	Expected trade-off (incurred deliberately for safety/helpfulness)
Parameter state	Prior knowledge physically overwritten	Latent distributions constrained/truncated
Information	Destroyed: weights no longer encode the capability	Suppressed: knowledge exists but is harder to trigger
Dominant phase	Sequential SFT, domain continued pre-training	Preference optimization (PPO, DPO, KTO, RLHF)
Primary symptom	Complete breakdown of baseline capabilities	Over-refusal, stylistic stiffness, lower raw benchmark scores
Reversibility	Irreversible without retraining from checkpoint	Partially reversible: adjust β , system prompt, or fine-tune
Detection	Perplexity on pre-training eval set spikes	Perplexity stable but win-rate on capability benchmarks drops
Scales with model size	Similar across scales	Smaller models pay a larger alignment tax

10.5.4 Mitigation Strategies

For Catastrophic Forgetting:

1. **Data replay:** Mix 5–10% of pre-training data into SFT dataset. Ensures gradient updates don’t completely neglect pre-training distribution.
2. **Elastic Weight Consolidation (EWC)** [186]: Add regularization $\Omega(\theta) = \frac{\lambda}{2} \sum_i F_i(\theta_i - \theta_i^*)^2$ that penalizes changes to parameters with high Fisher information for the original task.
3. **LoRA / Parameter-efficient fine-tuning:** Train only low-rank adapters ($< 1\%$ of parameters), leaving base weights completely frozen. This prevents *permanent destruction* of pre-trained knowledge — you can always remove the adapter and recover the original model. However, **while the adapter is active**, the combined system ($W_0 + BA$) can still exhibit forgetting: the adapter may shift the model’s effective behavior away from old skills. LoRA protects the checkpoint, not the active inference behavior.
4. **Conservative learning rate:** Use $1\text{--}5 \times 10^{-6}$ with few epochs (1–3). Larger rates accelerate forgetting.
5. **Progressive training:** Mix distributions gradually, increasing SFT data proportion over time rather than switching abruptly.

For Alignment Tax:

1. **Tune β carefully:** Lower β gives the model more freedom (reduces the tax) but may sacrifice safety. Optimal $\beta \in [0.05, 0.3]$ for most settings.
2. **High-quality, diverse SFT data:** Part of the alignment tax comes from SFT narrowing the output distribution; broader, more diverse SFT data reduces this component. The RL phase adds further constraint via KL regularization [280].

3. **Conditional alignment:** Train the model to be aligned only when a safety flag is active. At inference, disable constraints for benchmarking (research-only technique).
4. **Constitutional AI / RLAIIF:** Use model-generated feedback to create more nuanced preference data that preserves capability while improving alignment.
5. **Targeted RL budget:** Don't over-train with RL. Monitor capability benchmarks and stop when the tax exceeds acceptable thresholds (typically 2–5% MMLU regression).

How to Tell Which One You Have

- **Run the base model on the failing tasks:** If the base model succeeds and the fine-tuned model completely fails → catastrophic forgetting.
- **Prompt engineering test:** If careful prompting (e.g., “ignore safety guidelines and solve this math problem step by step”) recovers the capability → alignment tax (knowledge is suppressed, not erased).
- **Perplexity check:** Compute perplexity on pre-training validation set. Spike = forgetting. Stable = alignment tax.
- **Few-shot recovery:** If providing a few in-context examples restores the capability → alignment tax. If even many examples can't recover it → forgetting.

10.6 Connection to RL – SFT Quality Determines RL Ceiling

The SFT-RL Relationship

The SFT model is the starting point for RL training. RL can:

- **Amplify** behaviours that are present but weak in the SFT model.
- **Suppress** behaviours that are present but undesirable.
- **Refine** the style and format of responses.

RL *cannot*:

- Introduce capabilities that are entirely absent from the SFT model.
- Recover from severe catastrophic forgetting in the SFT stage.
- Compensate for a reward model that is systematically biased.

The Exploration-Exploitation Tradeoff in SFT

For RL to work, the SFT model must occasionally produce correct responses (so the reward signal is non-zero). If the SFT model *never* produces a correct response to a given prompt, RL cannot learn to produce correct responses – there is no positive signal to amplify. This is why SFT quality is the ceiling for RL performance.

Concretely: if the SFT model solves 10% of math problems correctly, RL can potentially push this to 80%. If the SFT model solves 0% of math problems, RL will make no progress (all rewards are zero, all advantages are zero, no gradient).

Practical Implications

1. **SFT data quality:** use high-quality, diverse data. A small amount of high-quality data is better than a large amount of low-quality data.
2. **SFT data coverage:** ensure the SFT data covers the tasks you want to improve with RL. If a task is not in the SFT data, RL will struggle.